

Containerized SonarQube + Snyk scanning — setup, status, blockers

Date: 2026-06-04 · **Program:** completeness program, Phase 2C **Author:** completeness-program agent · **Classification:** project-specific

This document describes the rootless, containerized SonarQube (static analysis + coverage aggregation) and Snyk (dependency + SAST) scanning IaC added to Lava, EXACTLY what ran on this host vs. what is blocked, and the gate-host / operator steps to complete each. It is deliberately honest per §6.I / §6.Z: nothing here claims a scan ran that did not run.

What was added

File	Purpose
<code>docker-compose.sonar.yml</code>	Rootless SonarQube CE + dedicated Postgres, bound to 127.0.0.1 only, named volumes, no privileged, no host sysctl.
<code>sonar-project.properties</code>	Sonar scanner config: Kotlin (app, core, feature) + Go (lava-api-go) sources; excludes submodules/, releases/, **/build/**, generated code; wires Detekt + Kover + Go coverage report paths.
<code>scripts/sonar-scan.sh</code>	Thin glue: auto-detect podman/docker (no sudo), up/down/status/scan, scanner via sonarsource/sonar-scanner-cli, SONAR_TOKEN from .env only.
<code>scripts/snyk-scan.sh</code>	Thin glue: containerized snyk/snyk snyk test + snyk code test; SNYK_TOKEN from .env only; exits (no fake pass) if absent.
<code>.snyk</code>	Ignore/patch policy (empty placeholders + reason/expiry convention).
<code>.env.example</code>	Adds SONAR_TOKEN, SNYK_TOKEN, SONAR_DB_* placeholders.

Constitutional posture

- **\$6.U — no sudo/su.** Neither the compose file nor the scripts invoke any privilege escalation. SonarQube's Elasticsearch `vm.max_map_count` requirement is NOT met by a `sysctl` tweak in our scripts; where the runtime VM/kernel value is below 262144, raising it is an operator/gate-host action OUTSIDE this repo.
- **\$6.R — no hardcoding.** Tokens are read from `.env` / environment only, never as literals in any tracked file.
- **\$6.J / \$6.Z — no bluff.** A scan runs only against a server that reports UP; a missing token or down server produces a clear non-zero exit, never a fabricated "passed" result.
- **Local-Only CI/CD.** Both scanners run on the developer's / gate-host's machine via rootless containers; no hosted-CI configuration is introduced.

What ran on THIS host (Darwin/arm64, rootless podman)

SonarQube — BROUGHT UP SUCCESSFULLY (status UP)

`./scripts/sonar-scan.sh up` pulled `sonarqube:community` (26.6.0.123539) + `postgres:16-alpine` and the server reached `{"status":"UP"}` in ~2 minutes.

The `vm.max_map_count` blocker did **not** materialize on this host: the podman applehv VM reports `/proc/sys/vm/max_map_count = 1048576` (well above the 262144 minimum), so Elasticsearch started cleanly with no root change. Evidence: `.lava-ci-evidence/completeness-program/sonar/2026-06-04/bring-up-2026-06-04.md`. The stack was torn down afterward (`./scripts/sonar-scan.sh down`).

The **scanner step was NOT run** here: it requires `SONAR_TOKEN`, which is generated in the Sonar UI after the first admin login (one-time interactive step). Server-up is proven; the scan is the operator's next action.

Snyk — BLOCKED on missing SNYK_TOKEN (honest refusal)

`./scripts/snyk-scan.sh deps` exited 1 with a clear "set `SNYK_TOKEN` in `.env`" message — no token, no scan, no fake pass. Evidence: `.lava-ci-evidence/completeness-program/snyk/2026-06-04/run-attempt-2026-06-04.md`.

How to run locally (rootless)

SonarQube

```
./scripts/sonar-scan.sh up          # start server + db
                                   (127.0.0.1 only)
./scripts/sonar-scan.sh status      # poll until {"status":"UP"}
# First time only: open http://127.0.0.1:9000 (admin/admin),
                                   change password,
# then My Account → Security → Generate Tokens. Put it in .env:
# SONAR_TOKEN=<generated-token>
./scripts/sonar-scan.sh scan        # analyze the workspace
./scripts/sonar-scan.sh down        # tear down
```

Snyk

```
# Put SNYK_TOKEN=<token> in .env (from https://app.snyk.io/
                                   account)
./scripts/snyk-scan.sh              # deps + SAST
./scripts/snyk-scan.sh deps         # dependency scan only
./scripts/snyk-scan.sh code         # SAST only
```

Blocked-on-this-host / operator-action summary

Item	Status	Action to complete
SonarQube server up	✅ ran (UP) on this host	none (worked)
vm.max_map_count ≥ 262144	✅ already 1048576 in this podman VM	On a host below 262144: operator raises it OUTSIDE this repo (NO sudo in our scripts). podman/ macOS: in the runtime VM; Linux: an operator-owned /etc/sysctl.d/99-sonarqube.conf drop-in.
SonarQube scan	⌚ operator-action	Generate SONAR_TOKEN in the UI, set in .env, run sonar-scan.sh scan.
Detekt / Kover / Go coverage reports	⌚ sibling-stream-dependent	Produced by Phase 2A (Detekt + Kover) and Phase 2B (Go coverage); sonar-project.properties references their expected paths and the scanner skips absent reports.
Snyk scan	⌚ operator-action	Set SNYK_TOKEN in .env, run snyk-scan.sh.

ci.sh wiring (recommendation — NOT done here per task scope)

scripts/ci.sh is intentionally NOT modified by this stream. When wired, both scanners should be hooked as **token-gated, opt-in security gates** in the `--full` path (e.g. behind a `--security` flag or a presence check on `SONAR_TOKEN` / `SNYK_TOKEN`): skip-with-notice when the token/server is absent (so default offline runs aren't broken), run when present. They **MUST** stay non-blocking-on-absent-token but blocking-on-found-high-severity once enabled, to preserve the anti-bluff posture (a security gate that silently passes because it never ran is itself a bluff).